第4章 从一条记录说起——InnoDB记录存储结构

4.1 准备工作

到现在为止，MySQL 对于我们来说还是一个“黑盒”，我们只负责使用客户端发送请求并等待服务器返回结果。表中的数据到底存到了哪里？以什么格式存放的？ MySQL 以什么方式来访问这些数据？这些问题的答案我们统统不知道。

我们前面在唠叨请求处理过程的时候提到，MySQL 服务器中负责对表中的数据进行读取和写入工作的部分是存储引擎，而服务器又支持不同类型的存储引擎，比如 InnoDB、MyISAM、MEMORY 啥的。不同的存储引擎一般是由不同的人为实现不同的特性而开发的，真实数据在不同存储引擎中的存放格式一般是不同的，甚至有的存储引擎（比如 MEMORY）都不用磁盘来存储数据。也就是对于使用 MEMORY 存储引擎的表来说，关闭服务器后表中的数据就消失了。由于 InnoDB 是 MySQL 默认的存储引擎，也是我们最常用到的存储引擎，另外我们也没有那么多时间去把各个存储引擎的内部实现都看一遍，所以本章要唠叨的是使用 InnoDB 作为存储引擎的记录存储结构。在了解了一个存储引擎的记录存储结构之后，其他的存储引擎都是“依葫芦画瓢”，就不多唠叨了。

4.2 InnoDB 页简介

InnoDB 是一个将表中的数据存储到磁盘上的存储引擎，即使我们关闭并重启服务器，数据还是存在。而真正处理数据的过程发生在内存中，所以需要把磁盘中的数据加载到内存中。如果是处理写入或修改请求，还需要把内存中的内容刷新到磁盘上。而我们知道读写磁盘的速度非常慢，与读写内存差了几个数量级。当我们想从表中获取某些记录时，InnoDB 存储引擎需要一条一条地把记录从磁盘上读出来么？不，那样会慢死，InnoDB 采取的方式是，将数据划分为若干个页，以页作为磁盘和内存之间交互的基本单位。InnoDB 中页的大小一般为 16KB。也就是在一般情况下，一次最少从磁盘中读取 16KB 的内容到内存中，一次最少把内存中的 16KB 内容刷新到磁盘中。

小贴士

系统变量 innodb_page_size 表明了 InnoDB 存储引擎中的页大小，默认值为 16384（单位是字节），也就是 16KB。该变量只能在第一次初始化 MySQL 数据目录时指定，之后就再也不能更改了（通过命令 mysqld --initialize 来初始化数据目录。我们之前没有过多地唠叨初始化数据目录的过程，大家只要知道在服务器运行过程中不可以更改页面大小就好了）。

4.3 InnoDB 行格式

我们平时都是以记录为单位向表中插入数据的，这些记录在磁盘上的存放形式也被称为行格式或者记录格式。设计 InnoDB 存储引擎的大叔到现在为止设计了 4 种不同类型的行格式，分别是 COMPACT、REDUNDANT、DYNAMIC 和 COMPRESSED。随着时间的推移，他们可能会设计出更多的行格式，但是不管怎么变，这些行格式在原理上大体都是相同的。

4.3.1 指定行格式的语法

我们可以在创建或修改表的语句中指定记录所使用的行格式：

```
CREATE TABLE 表名 (列的信息) ROW_FORMAT=行格式名称;

ALTER TABLE 表名 ROW_FORMAT=行格式名称;
```

比如在 xiaohaizi 数据库中创建一个演示用的表 record_format_demo，可以这样指定它的行格式：

```
mysql> USE xiaohaizi;
Database changed

mysql> CREATE TABLE record_format_demo (
    ->     c1 VARCHAR(10),
    ->     c2 VARCHAR(10) NOT NULL,
    ->     c3 CHAR(10),
    ->     c4 VARCHAR(10)
    -> ) CHARSET=ascii ROW_FORMAT=COMPACT;
Query OK, 0 rows affected (0.03 sec)
```

可以看到，我们刚刚创建的这个表的行格式就是 COMPACT。另外，我们还显式指定了这个表的字符集为 ascii。因为 ascii 字符集只包括空格、标点符号、数字、大小写字母和一些不可见字符，所以汉字是不能存到这个表里的。向这个表中插入两条记录：

```
mysql> INSERT INTO record_format_demo(c1, c2, c3, c4) VALUES ('aaaa', 'bbb', 'cc', 'd'),
('eeee', 'fff', NULL, NULL);
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

现在，表中的记录就是这个样子的：

```
mysql> SELECT * FROM record_format_demo;
+------+-----+------+------+
| c1   | c2  | c3   | c4   |
+------+-----+------+------+
| aaaa | bbb | cc   | d    |
| eeee | fff | NULL | NULL |
+------+-----+------+------+
2 rows in set (0.00 sec)
```

演示表的内容也填充好了，现在来看看各个行格式下的存储结构到底有啥不同。

4.3.2 COMPACT 行格式

话不多说，直接看图 4-1。

图 4-1 COMPACT 行格式示意图

从图 4-1 中可以看出，一条完整的记录其实可以被分为记录的额外信息和记录的真实数据两大部分。下面我们分别看一下这两大部分的组成。

1. 记录的额外信息

这部分信息是服务器为了更好地管理记录而不得不额外添加的一些信息。这些额外信息分为 3 个部分，分别是变长字段长度列表、NULL 值列表和记录头信息。

（1）变长字段长度列表

我们知道，MySQL 支持一些变长的数据类型，比如 VARCHAR(M)、VARBINARY(M)、各种 TEXT 类型、各种 BLOB 类型。我们也可以把拥有这些数据类型的列称为变长字段。变长字段中存储多少字节的数据是不固定的，所以我们在存储真实数据的时候需要顺便把这些数据占用的字节数也存起来，这样才不至于把 MySQL 服务器搞懵。也就是说这些变长字段占用的存储空间分为两部分：

- 真正的数据内容；
- 该数据占用的字节数。

在 COMPACT 行格式中，所有变长字段的真实数据占用的字节数都存放在记录的开头位置，从而形成一个变长字段长度列表，各变长字段的真实数据占用的字节数按照列的顺序逆序存放。再次强调一遍，是逆序存放！

小贴士

关于为啥逆序存放会在下一章介绍，这里少安毋躁。

我们拿 record_format_demo 表中的第一条记录来举个例子。因为 record_format_demo 表的 c1、c2、c4 列都是 VARCHAR(10) 类型的，也就是变长的数据类型，所以这 3 个列的值占用的存储空间字节数都需要保存在记录开头处。record_format_demo 表中的各个列使用的都是 ascii 字符集，每个字符只需要一个字节来编码。来看一下第一条记录各变长字段内容的长度（见表 4-1）。

表 4-1 第一条记录中各变长字段内容的长度

列名	存储内容	内容长度（十进制表示）	内容长度（十六进制表示）
c1	'aaaa'	4	0x04
c2	'bbb'	3	0x03
c4	'd'	1	0x01

因为这些长度值需要按照列的顺序逆序存放，所以最后变长字段长度列表的字节串用十六

进制表示的效果就是：

```
01 03 04
```

需要说明的是，上述各个字节之间实际上没有空格，这里使用空格只是为了方便理解。

把这个字节串组成的变长字段长度列表填入图 4-1 中的效果如图 4-2 所示。

图 4-2 第一条记录的存储格式

由于第一条记录中 c1、c2、c4 列中的字符串都比较短，也就是说占用的字节数比较小（c1 列内容是 'aaaa'，占用 4 字节；c2 列内容是 'bbb'，占用 3 字节；c4 列内容是 'd'，占用 1 字节），每个变长字段的内容占用的字节数用 1 字节就可以表示（也就是 4、3、1 这 3 个数字可以分别用字节 0x04、0x03、0x01 表示）。但是，如果变长字段的内容占用的字节数比较多，可能就需要用 2 字节来表示。至于用 1 字节还是 2 字节来表示变长字段的真实数据占用的字节数，InnoDB 有它的一套规则。为了更好地表述清楚这个规则，我们引入 W、M 和 L 这几个符号，先分别看看这些符号的意思。

- 假设某个字符集中最多需要 W 字节来表示一个字符（也就执行 SHOW CHARSET 语句后结果中的 Maxlen 列）。比如 utf8mb4 字符集中的 W 就是 4，utf8 字符集中的 W 就是 3，gbk 字符集中的 W 就是 2，ascii 字符集中的 W 就是 1。
- 对于变长类型 VARCHAR(M) 来说，这种类型表示能存储最多 M 个字符（注意是字符不是字节），所以这种类型能表示的字符串最多占用的字节数就是 M × W。
- 假设该变长字段实际存储的字符串占用的字节数是 L。

确定使用 1 字节还是 2 字节来表示一个变长字段的真实数据占用的字节数的规则就是这样：

- 如果 M × W ≤ 255，那么使用 1 字节来表示真实数据占用的字节数。

小贴士

InnoDB 在读取记录的变长字段长度列表时先查看表结构，先查看表结构，先查看表结构（重要的事情说三遍）。如果某个变长字段允许存储的最大字节数不大于 255，可以认为只使用 1 字节来表示真实数据占用的字节数。

- 如果 M × W > 255，则分为下面两种情况：
  - 如果 L ≤ 127，则用 1 字节来表示真实数据占用的字节数；
  - 如果 L > 127，则用 2 字节来表示真实数据占用的字节数。

小贴士

InnoDB 在读取记录的变长字段长度列表时先查看表结构。如果某个变长字段允许存储的最大字节数大于 255，该怎么区分它正在读的某个字节是一个单独的字段长度还是半个字段长度呢？设计 InnoDB 的大叔使用该字节的第一个二进制位作为标志位：如果该字节的第一个位为 0，该字节就是一个单独的字段长度（在使用一个字节表示不大于 127 的数字时，第一个位都为 0）；如果该字节的第一个位为 1，该字节就是半个字段长度。这个规则特别像我们前面说过的 GBK 字符集的编码规则。

对于一条记录来说，如果某个字段占用的字节数特别多，InnoDB 有可能把该字段的值的一部分数据存放到所谓的溢出页中（我们后面会详细唠叨）。那么该字段在记录的变长字段长度列表处只存储留在本页面中的长度，所以使用 2 字节就可以表示这个留在本页面中的字节长度。尽管也是使用 2 字节，但对于溢出字段来说，采用的方案并不是单纯地将首字节的第一个二进制位置为 0，而是采用了一种特殊的表示方式。关于表示溢出字段占用字节数的特殊表示方式我们就不多唠叨了，这里就是提一下，大家也不用深究。

总结一下就是：如果该变长字段允许存储的最大字节数（M × W）超过 255 字节，并且真实数据占用的字节数（L）超过 127 字节，则使用 2 字节来表示真实数据占用的字节数，否则使用 1 字节。

另外需要注意的一点是，变长字段长度列表中只存储值为非 NULL 的列的内容长度，不存储值为 NULL 的列的内容长度。也就是说对于第二条记录，因为 c4 列的值为 NULL，所以第二条记录的变长字段长度列表只需要存储 c1 和 c2 列的内容长度即可。其中 c1 列存储的值为 'eeee'，占用的字节数为 4；c2 列存储的值为 'fff'，占用的字节数为 3。数字 4 可以用 1 字节（0x04）表示，3 也可以用 1 字节（0x03）表示，这样第二条记录的整个变长字段长度列表共需 2 字节。填充完变长字段长度列表的两条记录的对比如图 4-3 所示。

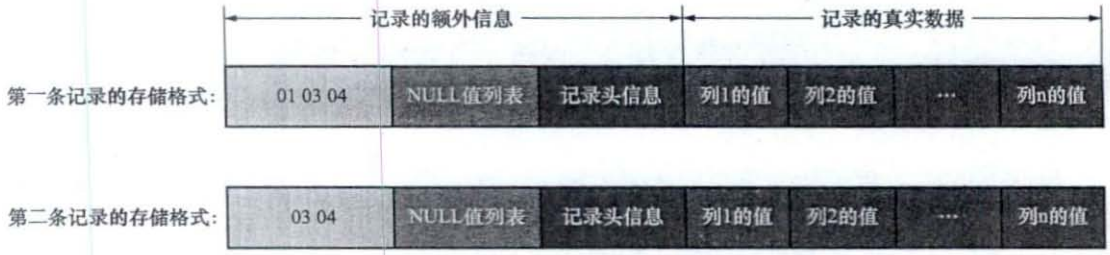

图 4-3 两条记录存储格式对比

小贴士

并不是所有记录都有这个变长字段长度列表部分，如果表中所有的列都不是变长的数据类型或者所有列的值都是 NULL 的话，就不需要有变长字段长度列表。

（2）NULL 值列表

我们知道，一条记录中的某些列可能存储 NULL 值，如果把这些 NULL 值都放到记录的真实数据中存储会很占地方，所以 COMPACT 行格式把一条记录中值为 NULL 的列统一管理起来，存储到 NULL 值列表中。它的处理过程如下所示。

1. 首先统计表中允许存储 NULL 的列有哪些。

   主键列以及使用 NOT NULL 修饰的列都是不可以存储 NULL 值的，所以在统计的时候不会把这些列算进去。比如表 record_format_demo 的 3 个列 c1、c3、c4 都允许存储 NULL 值，而 c2 列使用 NOT NULL 进行了修饰，不允许存储 NULL 值。

2. 如果表中没有允许存储 NULL 的列，则 NULL 值列表也就不存在了，否则将每个允许存储 NULL 的列对应一个二进制位，二进制位按照列的顺序逆序排列。二进制位表示的意义如下：

   - 二进制位的值为 1 时，代表该列的值为 NULL；

- 二进制位的值为 0 时，代表该列的值不为 NULL。

因为表 record_format_demo 有 3 个值允许为 NULL 的列，所以这 3 个列和二进制位的对应关系如图 4-4 所示。

再一次强调，二进制位按照列的顺序逆序排列，所以第一个列 c1 和最后一个二进制位对应。

3. MySQL 规定 NULL 值列表必须用整数个字节的位表示，如果使用的二进制位个数不是整数个字节，则在字节的高位补 0。

表 record_format_demo 只有 3 个值允许为 NULL 的列，对应 3 个二进制位，不足一个字节，所以在字节的高位补 0，效果如图 4-5 所示。

图 4-4　列和二进制位的对应关系

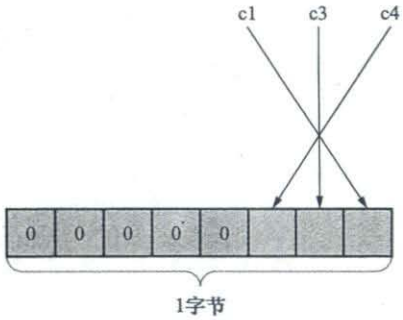

图 4-5　字节高位补 0 的效果

依此类推，如果一个表中有 9 个值允许为 NULL 的列，则这个记录的 NULL 值列表部分就需要 2 字节来表示了。

知道了规则之后，我们再返回头看看表 record_format_demo 中两条记录中的 NULL 值列表应该怎么储存。因为只有 c1、c3、c4 这 3 个列允许存储 NULL 值，所以记录的 NULL 值列表处只需要一个字节。

- 对于第一条记录来说，c1、c3、c4 这 3 个列的值都不为 NULL，所以它们对应的二进制位都是 0，如图 4-6 所示。

  所以第一条记录的 NULL 值列表用十六进制表示就是 0x00。

- 对于第二条记录来说，c1、c3、c4 这 3 个列中 c3 和 c4 的值都为 NULL，所以这 3 个列对应的二进制位的情况如图 4-7 所示。

  所以第二条记录的 NULL 值列表用十六进制表示就是 0x06。

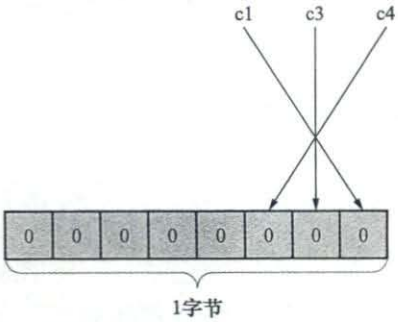

图 4-6　第一条记录的 c1、c3、c4 三个列的值

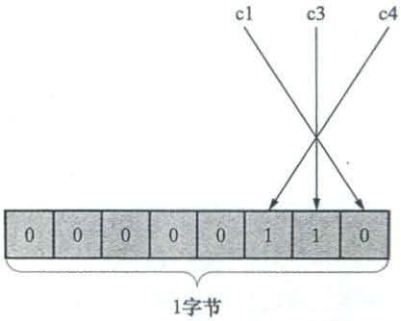

图 4-7　第二条记录的 c1、c3、c4 三个列的值

这两条记录在填充了 NULL 值列表后的示意图如图 4-8 所示。

图 4-8 两条记录在填充了 NULL 值列表后的示意图

（3）记录头信息

除了变长字段长度列表、NULL 值列表之外，还有一个称之为记录头信息的部分。记录头信息由固定的 5 字节组成，用于描述记录的一些属性。5 字节也就是 40 个二进制位，不同的位代表不同的意思，如图 4-9 所示。

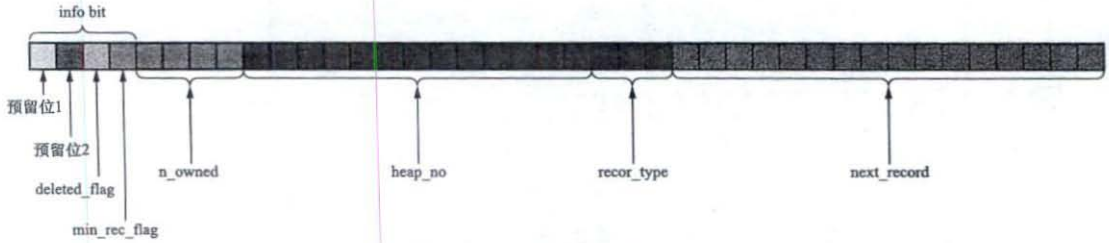

图 4-9 记录头信息示意图

这些二进制位代表的详细信息如表 4-2 所示。

表 4-2 记录头信息中各二进制位代表的详细信息

名称	大小（位）	描述
预留位 1	1	没有使用
预留位 2	1	没有使用
deleted_flag	1	标记该记录是否被删除
min_rec_flag	1	B+ 树的每层非叶子节点中最小的目录项记录都会添加该标记
n_owned	4	一个页面中的记录会被分成若干个组，每个组中有一个记录是“带头大哥”，其余的记录都是“小弟”。“带头大哥”记录的 n_owned 值代表该组中所有的记录条数，“小弟”记录的 n_owned 值都为 0
heap_no	13	表示当前记录在页面堆中的相对位置
record_type	3	表示当前记录的类型，0 表示普通记录，1 表示 B+ 树非叶子节点的目录项记录，2 表示 Infimum 记录，3 表示 Supremum 记录
next_record	16	表示下一条记录的相对位置

另外，记录头信息的前 4 个位也被称为 info bit。

大家千万不要被这么多属性和陌生的概念吓着，这里把这些位代表的意思都写出来只是为了内容的完整性。没必要把它们的意思都记住，记住也没啥用，现在只需要看一遍混个脸熟，等之后用到这些属性的时候再回过头来看就好了。

因为我们并不清楚这些属性的详细用法，所以这里就不分析各个属性值是怎么产生的了，之后我们会详细唠叨的，少安毋躁。现在直接看一下 record_format_demo 表中的两条记录的记录头信息分别是什么，如图 4-10 所示。

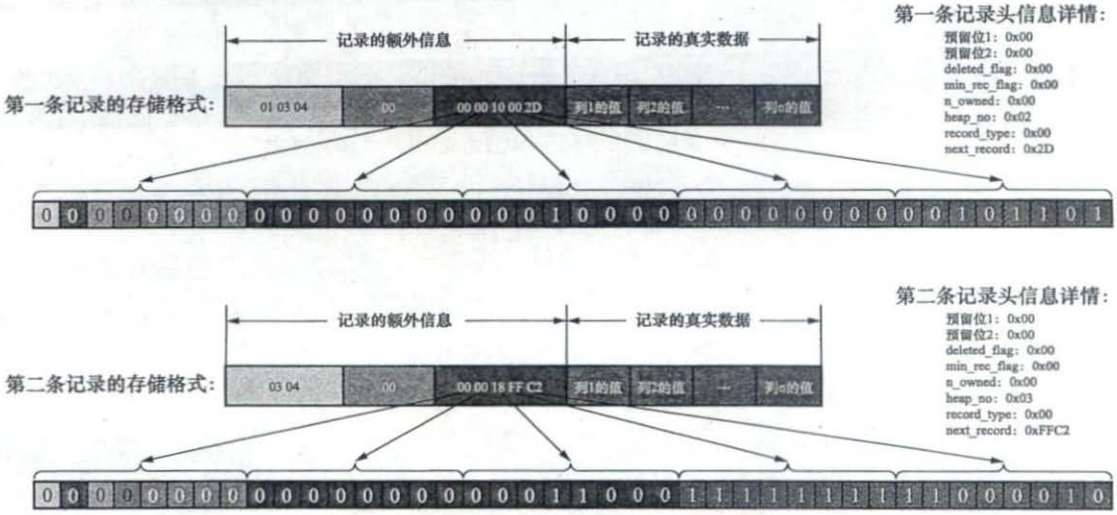

图 4-10　两条记录的记录头信息详情

小贴士

再一次强调，大家如果看不懂记录头信息中各个位代表的概念也千万别纠结，我们后面会说的。

2. 记录的真实数据

对于 record_format_demo 表来说，记录的真实数据除了 c1、c2、c3、c4 这几个我们自己定义的列的数据外，MySQL 会为每个记录默认地添加一些列（也称为隐藏列），具体的列如表 4-3 所示。

表 4-3　MySQL 为每个记录默认添加的列

列名	是否必需	占用空间	描述
row_id	否	6 字节	行 ID，唯一标识一条记录
trx_id	是	6 字节	事务 ID
roll_pointer	是	7 字节	回滚指针

小贴士

实际上这几个列的真正名称是 DB_ROW_ID、DB_TRX_ID、DB_ROLL_PTR，只不过我觉得它们不好看，才写成了小写形式，大家之后在阅读文档或者别的资料时意识到这个问题就好了。

这里需要提一下 InnoDB 表的主键生成策略：优先使用用户自定义的主键作为主键；如果用户没有定义主键，则选取一个不允许存储 NULL 值的 UNIQUE 键作为主键；如果表中连不允许存储 NULL 值的 UNIQUE 键都没有定义，则 InnoDB 会为表默认添加一个名为 row_id 的隐藏列作为主键。

所以从表 4-3 中可以看出，InnoDB 存储引擎会为每条记录都添加 trx_id 和 roll_pointer 这两个列，但是 row_id 是可选的（在没有自定义主键以及不允许存储 NULL 值的 UNIQUE 键的

情况下才会添加该列）。这些隐藏列的值不用我们操心，InnoDB 存储引擎会自动帮我们生成。

因为表 record_format_demo 并没有定义主键，所以 MySQL 服务器会为每条记录增加上述的 3 个列。现在看一下加上记录的真实数据的两条记录长什么样子，如图 4-11 所示。

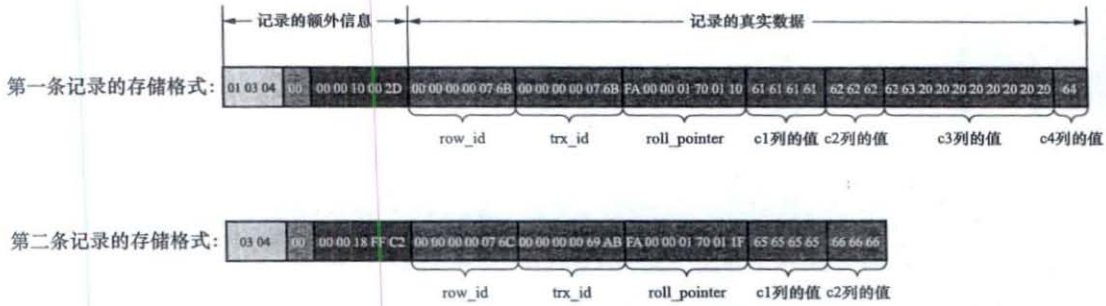

图 4-11 记录真实数据的两条记录

看图 4-11 时要注意以下几点。

- 表 record_format_demo 使用的是 ascii 字符集，所以 0x61616161 就表示字符串 'aaaa'，0x626262 就表示字符串 'bbb'；依此类推。
- 注意第一条记录中 c3 列是 CHAR(10) 类型的，它实际存储的字符串是 'cc'，使用 ascii 字符集来编码这个字符串得到的结果是 '0x6363'。虽然表示这个字符串只占用了 2 字节，但整个 c3 列仍然占用了 10 字节的空间，除真实数据以外的 8 字节统统都用空格字符填充，空格字符在 ascii 字符集中的编码就是 0x20。
- 注意第二条记录中 c3 和 c4 列的值都为 NULL，它们被存储在了前面的 NULL 值列表处，在记录的真实数据处就不再冗余存储，从而节省了存储空间。

3. CHAR(M) 列的存储格式

前面讲到，在 COMPACT 行格式下，变长字段长度列表只是用来存放一条记录中各个变长字段的值占用的字节长度的。record_format_demo 表的 c1、c2、c4 列的类型是 VARCHAR(10)，也就是说 c1、c2、c4 都是变长字段；而 c3 列的类型是 CHAR(10)，也就是说 c3 列不属于变长字段。所以会把一条记录的 c1、c2、c4 这 3 个列占用的字节长度逆序存到变长字段长度列表中，如图 4-12 所示。

但这只是建立在我们的 record_format_demo 表采用的是 ascii 字符集的情况下，这个字符集采用固定的一个字节来编码一个字符，是一个定长编码字符集。如果采用变长编码的字符集（也就是表示一个字符需要的字节数不确定，比如 gbk 表示一个字符要 1 ～ 2 字节、utf8 表示一个字符要 1 ～ 3 字节等），虽然 c3 列的类型是 CHAR(10)，但是设计 COMPACT 行格式的大叔规定，此时该列的值占用的字节数也会被存储到变长字段长度列表中。比如我们修改一下 c3 列的字符集：

```
mysql> ALTER TABLE record_format_demo MODIFY COLUMN c3 CHAR(10) CHARACTER SET utf8;
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

修改该列字符集后，记录的变长字段长度列表也发生了变化，如图 4-13 所示。

这就意味着，对于 CHAR(M) 类型的列来说，当列采用的是定长编码的字符集时，该列占用的字节数不会被加到变长字段长度列表；而如果采用变长编码的字符集时，该列占用的字节数就会被加到变长字段长度列表。

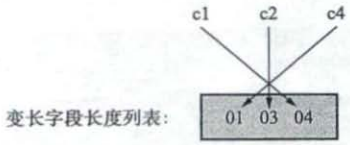

图 4-12　变长字段长度列表

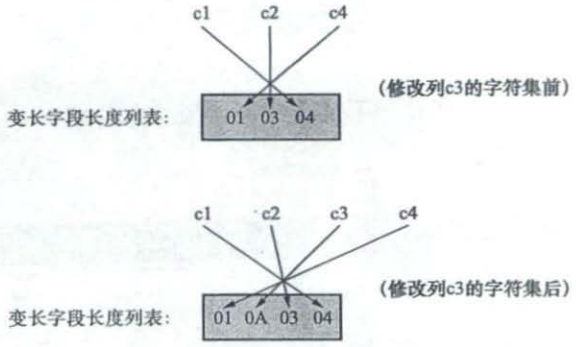

图 4-13　变长字段长度列表的变化

另外还有一点需要注意，设计 COMPACT 行格式的大叔还规定，采用变长编码字符集的 CHAR(M) 类型的列要求至少占用 M 个字节，而 VARCHAR(M) 却没有这个要求。比如对于使用 utf8 字符集、类型为 CHAR(10) 的列来说，该列存储的数据占用的字节长度的范围就是 10 ～ 30 字节。即使我们向该列中存储一个空字符串也会占用 10 字节，这主要是希望在将来更新该列时，在新值的字节长度大于旧值的字节长度但不大于 10 个字节时，可以在该记录处直接更新。而不是在存储空间中再重新分配一个新的记录空间，导致原有的记录空间成为所谓的碎片（大家应该在这里感受到设计 COMPACT 行格式的大叔既想节省存储空间，又不想因为更新 CHAR(M) 类型的列而产生碎片的纠结心情了吧）。

4.3.3　REDUNDANT 行格式

其实知道了 COMPACT 行格式之后，其他的行格式就是“依葫芦画瓢”了。我们现在要介绍的 REDUNDANT 行格式是 MySQL 5.0 之前就在使用的一种行格式。也就是说它已经非常古老了，但是本着知识完整性还是要提一下，大家看一下就好。

REDUNDANT 行格式的全貌如图 4-14 所示。

图 4-14　REDUNDANT 行格式示意图

现在把表 record_format_demo 的行格式修改为 REDUNDANT：

```
mysql> ALTER TABLE record_format_demo ROW_FORMAT=REDUNDANT;
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

为了方便大家理解和节省篇幅，我们直接把表 record_format_demo 在 REDUNDANT 行格式下的两条记录的具体格式提供出来（见图 4-15），之后再着重分析两种行格式的不同即可。

下边我们从各个方面看一下 REDUNDANT 行格式与 COMPACT 行格式相比，有什么不同的地方。

1. 字段长度偏移列表

注意 COMPACT 行格式的开头是变长字段长度列表，而 REDUNDANT 行格式的开头是字

段长度偏移列表，它与变长字段长度列表相比有两处不同：

- 没有了“变长”两个字，意味着 REDUNDANT 行格式会把该条记录中所有列（包括隐藏列）的长度信息都按照逆序存储到字段长度偏移列表；
- 多了个“偏移”两个字，这意味着计算列值长度的方式不像 COMPACT 行格式那么直观，它是采用两个相邻偏移量的差值来计算各个列值的长度。

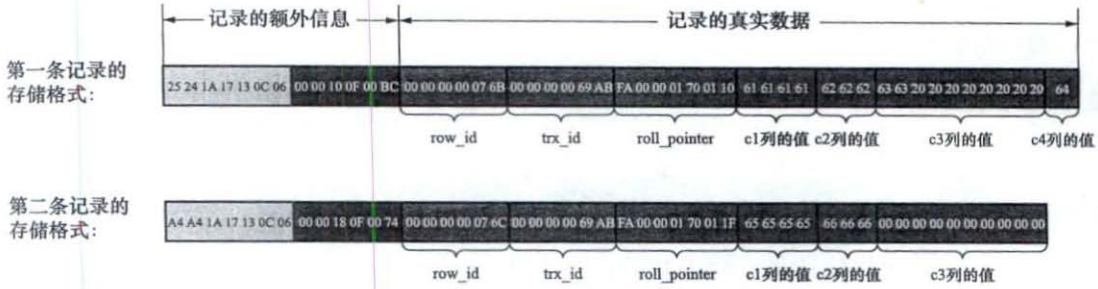

图 4-15 REDUNDANT 行格式下两条记录的具体格式

比如第一条记录的字段长度偏移列表就是：

```
25 24 1A 17 13 0C 06
```

因为它是逆序排放的，所以按照列的顺序排列就是：

```
06 0C 13 17 1A 24 25
```

按照两个相邻偏移量的差值来计算各个列值的长度的意思就是：

- 第一列（row_id）的长度就是 0x06 个字节，也就是 6 字节；
- 第二列（trx_id）的长度就是（0x0C - 0x06）个字节，也就是 6 字节；
- 第三列（roll_pointer）的长度就是（0x13 - 0x0C）个字节，也就是 7 字节；
- 第四列（c1）的长度就是（0x17 - 0x13）个字节，也就是 4 字节；
- 第五列（c2）的长度就是（0x1A - 0x17）个字节，也就是 3 字节；
- 第六列（c3）的长度就是（0x24 - 0x1A）个字节，也就是 10 字节；
- 第七列（c4）的长度就是（0x25 - 0x24）个字节，也就是 1 字节。

2. 记录头信息

REDUNDANT 行格式的记录头信息占用 6 字节，总计 48 个二进制位，这些二进制位代表的意思如表 4-4 所示。

表 4-4 REDUNDANT 行格式的记录头信息占用的 48 个二进制位及其描述

名称	大小（位）	描述
预留位 1	1	没有使用
预留位 2	1	没有使用
deleted_flag	1	标记该记录是否被删除
min_rec_flag	1	B+ 树的每层非叶子节点中的最小的目录项记录都会添加该标记

续表

名称	大小（位）	描述
n_owned	4	一个页面中的记录会被分成若干个组，每个组中有一个记录是“带头大哥”，其余的记录都是“小弟”。“带头大哥”记录的 n_owned 值代表该组中所有的记录条数，“小弟”记录的 n_owned 值都为 0
heap_no	13	表示当前记录在页面堆中的相对位置
n_field	10	表示记录中列的数量
1byte_offs_flag	1	标记字段长度偏移列表中每个列对应的偏移量是使用 1 字节还是 2 字节表示的
next_record	16	表示下一条记录的相对位置

第一条记录中的头信息是：

```
00 00 10 0F 00 BC
```

根据这 6 字节可以计算出各个属性的值，如下：

```
预留位1：0x00
预留位2：0x00
deleted_flag: 0x00
min_rec_flag: 0x00
n_owned: 0x00
heap_no: 0x02
n_field: 0x07
1byte_offs_flag: 0x01
next_record:0xBC
```

与 COMPACT 行格式的记录头信息对比来看，有两处不同：

- REDUNDANT 行格式多了 n_field 和 1byte_offs_flag 这两个属性；
- REDUNDANT 行格式没有 record_type 这个属性。

3. 记录头信息中的 1byte_offs_flag 的值是怎么选择的

从本质上来说，字段长度偏移列表存储的偏移量指的是每个列的值占用的空间在记录的真实数据处结束的位置。这句话有点儿拗口，我们还是拿 record_format_demo 第一条记录为例来分析一下。0x06 代表第一个列在记录的真实数据第 6 字节处结束，0x0C 代表第二个列在记录的真实数据第 12 字节处结束，0x13 代表第三个列在记录的真实数据第 19 字节处结束……最后一个列对应的偏移量值为 0x25，也就意味着最后一个列在记录的真实数据第 37 字节处结束，也就意味着整条记录的真实数据实际上占用 37 字节。

在字段长度偏移列表中，每个列对应的偏移量可以使用 1 字节或者 2 字节来存储，那到底什么时候使用 1 字节、什么时候用 2 字节呢？这是根据该条 REDUNDANT 行格式记录的真实数据占用的总大小来判断的。

- 当记录的真实数据占用的字节数不大于 127（十六进制 0x7F，二进制 01111111）时，每个列对应的偏移量占用 1 字节。

小贴士

如果整个记录的真实数据占用的存储空间都不大于 127 字节，那么每个列对应的偏移量值肯定也就不大于 127，也就可以使用 1 字节来表示偏移量。

- 当记录的真实数据占用的字节数大于 127，但不大于 32767（十六进制 0x7FFF，二进制 0111111111111111）时，每个列对应的偏移量占用 2 字节。
- 有没有记录的真实数据大于 32767 的情况呢？有，不过此时记录的一部分已经存放到了所谓的溢出页中（后面我们会详细讨论），在本页中只保留前 768 字节和 20 字节的溢出页面地址（当然这 20 字节中还记录了一些别的信息）。在这种情况下只使用 2 字节来存储每个列对应的偏移量就够了。

大家可以看出来，设计 REDUNDANT 行格式的大叔采用的策略还是比较简单粗暴的：直接使用整个记录的真实数据占用的字节长度来决定使用 1 字节还是 2 字节存储列对应的偏移量。只要整条记录的真实数据占用的存储空间长度大于 127 字节，即使某个列的值占用的存储空间不大于 127 字节，那对不起，也需要使用 2 字节来表示该列对应的偏移量。简单粗暴，就是这么简单粗暴（所以这种行格式有些过时了）。

小贴士

不知大家是否有疑问，既然一个字节表示的范围是 0 ～ 255，为啥在记录的真实数据占用的存储空间大于 127 字节时就采用 2 字节表示各个列的偏移量，而不是大于 255 字节时再采用 2 字节表示各个列的偏移量呢？少安毋躁，后面唠叨 NULL 值处理的时候会解决这个疑惑。

为了在解析记录时知道每个列的偏移量是使用 1 字节还是 2 字节表示的，设计 REDUNDANT 行格式的大叔特意在记录头信息中放置了一个称为 1byte_offs_flag 的属性：

- 当它的值为 1 时，表明使用 1 字节存储偏移量。
- 当它的值为 0 时，表明使用 2 字节存储偏移量。

4. REDUNDANT 行格式中 NULL 值的处理

因为 REDUNDANT 行格式并没有 NULL 值列表，所以设计 REDUNDANT 行格式的大叔在字段长度偏移列表中对各列对应的偏移量处做了一些特殊处理——将列对应的偏移量值的第一个比特位作为是否为 NULL 的依据，该比特位也可以称之为 NULL 比特位。也就是说在解析一条记录的某个列时，首先看一下该列对应的偏移量的 NULL 比特位是否为 1。如果为 1，那么该列的值就是 NULL，否则就不是 NULL。

这也就解释了前文提到的“为什么只要记录的真实数据大于 127（十六进制 0x7F，二进制 01111111）时，就采用 2 字节来表示一个列对应的偏移量”。原因就是第一个比特位是所谓的 NULL 比特位，用来标记该列的值是否为 NULL。

但是还有一点需要注意，对于值为 NULL 的列来说，该列的类型是否为变长类型决定了该列在记录的真实数据处的存储方式。我们接下来分析 record_format_demo 表的第二条记录，它对应的字段长度偏移列表如下：

```
A4 A4 1A 17 13 0C 06
```

按照列的顺序排放就是：

```
06 C 13 17 1A A4 A4
```

我们分情况看一下。

- 如果存储 NULL 值的字段是定长类型的，比如是 CHAR(M) 数据类型的，则 NULL 值也将占用记录的真实数据部分，并把该字段对应的数据使用 0x00 字节填充。

在图 4-15 所示的第二条记录中，c3 列的值是 NULL，而 c3 列的类型是 CHAR(10)，在记录的真实数据部分占用了 10 字节，所以我们看到在 REDUNDANT 行格式中使用 0x00000000000000000000 来表示 NULL 值。

另外，c3 列对应的偏移量为 0xA4，对应的二进制为 10100100，可以看到最高位为 1，意味着该列的值是 NULL。将最高位去掉后的值变成了 0100100，对应的十进制值为 36，而 c2 列对应的偏移量为 0x1A，也就是十进制的 26。36－26＝10，也就是说最终 c3 列占用的存储空间为 10 字节。

- 如果存储 NULL 值的字段是变长数据类型的，则不在记录的真实数据部分占用任何存储空间。

比如 record_format_demo 表的 c4 列是 VARCHAR(10) 类型的，VARCHAR(10) 是一个变长数据类型。c4 列对应的偏移量为 0xA4，与 c3 列对应的偏移量相同。这也就意味着它的值也为 NULL，将 0xA4 的最高位去掉后对应的十进制值也是 36。36 － 36 ＝ 0，也就意味着 c4 列本身不占用记录的真实数据处的空间。

除了上面几点之外，REDUNDANT 行格式和 COMPACT 行格式大致相同，但是能明显感觉到使用 COMPACT 行格式的记录占用的空间更少一点，所以显得更紧凑，这样就可以在一个页面中存放尽可能多的记录。

5. CHAR(M) 列的存储格式

我们知道，在使用 COMPACT 行格式时，CHAR(M) 类型的列所使用的字符集不同（具体分为定长编码的字符集和变长编码的字符集），该列的真实数据的具体存储方案也不同。

而 REDUNDANT 行格式则十分干脆，不管该列使用的字符集是啥，只要使用 CHAR(M) 类型，该列的真实数据占用的存储空间大小就是该字符集表示一个字符最多需要的字节数和 M 的乘积。比如，使用 utf8 字符集的 CHAR(10) 类型的列，其真实数据占用的存储空间大小始终为 30 字节；使用 gbk 字符集的 CHAR(10) 类型的列，其真实数据占用的存储空间大小始终为 20 字节。这样的话，将来在对该列进行更新时，可以直接在原位置更新，而不需要为记录申请新的存储空间。当然，这样的坏处就是可能会浪费一些存储空间。

4.3.4 溢出列

1. 溢出列

我们以使用 ascii 字符集的 off_page_demo 表为例，向该表中插入一条记录：

```
mysql> CREATE TABLE off_page_demo (
    ->       c VARCHAR(65532)
    -> ) CHARSET=ascii ROW_FORMAT=COMPACT;
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO off_page_demo (c) VALUES(REPEAT('a', 65532));
Query OK, 1 row affected (0.00 sec)
```

其中 REPEAT('a', 65532) 是一个函数调用，它表示生成一个把字符 'a' 重复 65532 次的字符串。

由于使用的是 ascii 字符集，所以这个字符串实际占用的字节数就是 65532。前文说过，InnoDB 中磁盘和内存交互的基本单位是页，也就是说 InnoDB 是以页为基本单位来管理存储空间的，我们的记录都会被分配到某个页中存储。而一个页的大小一般是 16KB，也就是 16384 字节，而本例中一个列的实际数据就需要占用 65532 字节，很显然一个页也存不了一条记录，这不是贼尴尬么。

在 COMPACT 和 REDUNDANT 行格式中，对于占用存储空间非常多的列，在记录的真实数据处只会存储该列的一部分数据，而把剩余的数据分散存储在几个其他的页中，然后在记录的真实数据处用 20 字节存储指向这些页的地址（当然，这 20 字节还包括分散在其他页面中的数据所占用的字节数），从而可以找到剩余数据所在的页，如图 4-16 所示。

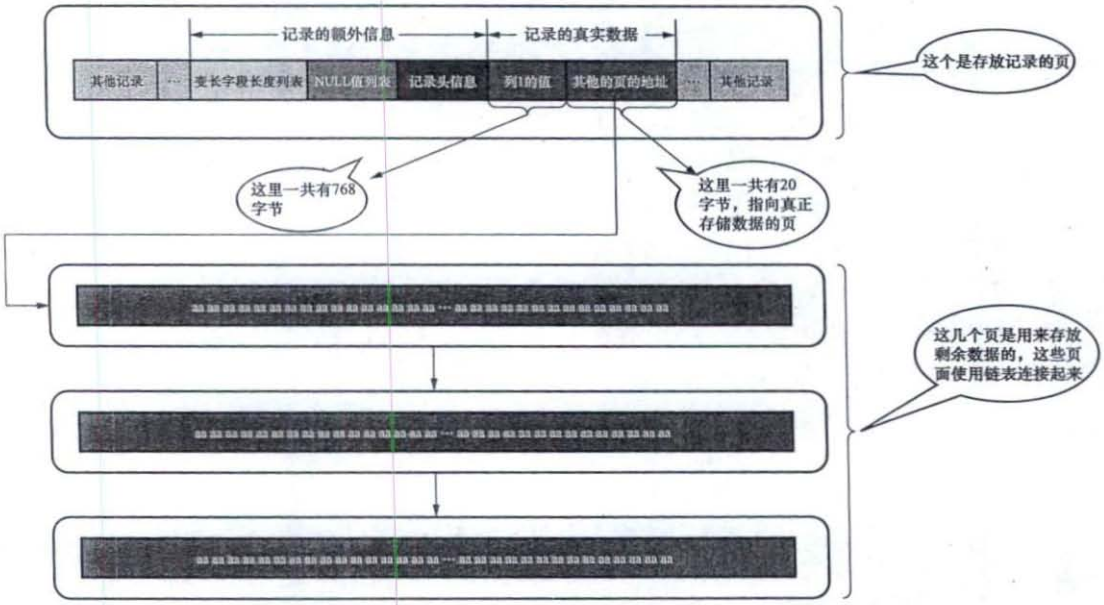

图 4-16 当列占用的存储空间相当大时，COMPACT 行格式的存储方式

从图 4-16 中可以看出，对于 COMPACT 和 REDUNDANT 行格式来说，如果某一列中的数据非常多，则在本记录的真实数据处只会存储该列前 768 字节的数据以及一个指向其他页的地址，然后把剩下的数据存放到其他页中。这些存储 768 字节之外的数据的页面也称为溢出页。图 4-17 是图 4-16 的简化示意图。

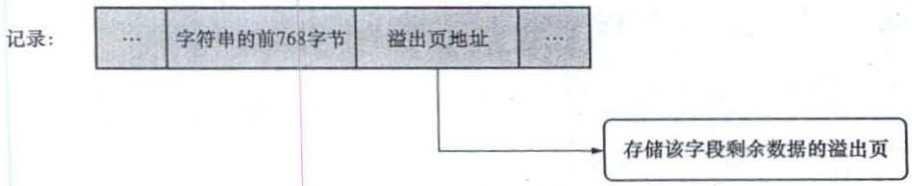

图 4-17 图 4-16 的简化示意图

off_page_demo 表的这条记录的列 c 的数据需要使用溢出页来存储，那么我们就把这个列称为溢出列（其实设计 InnoDB 的大叔把该列称之为 off-page 列）。最后需要注意的是，不只是 VARCHAR(M) 类型的列可能成为溢出列，像 TEXT、BLOB 这些类型的列在存储的数据相当多的时候也会成为溢出列。

2. 产生溢出页的临界点

一个列在存储了多少字节之后会变为溢出列呢？

MySQL 中规定一个页中至少存放两行记录。至于为什么这么规定后面再说，现在看一下这个规定造成的影响。以上面的 off_page_demo 表为例，它只有一个列 c。我们往这个表中插入两条记录，每条记录最少包含多少字节的数据才会需要溢出页呢？这得分析一下页中的空间都是如何利用的。

- 每个页除了存放我们的记录以外，也需要存储一些额外的信息。这些乱七八糟的额外信息加起来需要 132 字节的空间（现在只要知道这个数字就好了，下一章可以精确计算），其他的空间都可以被用来存储记录。
- 每个记录需要的额外信息是 27 字节。这 27 字节包括下面这些内容：
  - 2 字节用于存储真实数据的长度；
  - 1 字节用于存储列是否是 NULL 值；
  - 5 字节大小的头信息；
  - 6 字节的 row_id 列；
  - 6 字节的 trx_id 列；
  - 7 字节的 roll_pointer 列。

假设一个列的真实数据占用的字节数为 n，设计 MySQL 的大叔规定，如果该列不发生溢出现象，就需要满足下面这个不等式：

$$132 + 2 \times (27 + n) < 16384$$

小贴士

有读者可能会有疑问，为啥上面的这个不等式不是 $132 + 2 \times (27 + n) \leqslant 16384$ 呢？我只能说设计 MySQL 的大叔就是这么规定的。另外，存放正常记录的页面和溢出页是两种不同类型的页面（下一章会介绍页面类型）。对于溢出页来说，并没有规定一个页面中最少存放两条记录。

通过求解这个不等式，得出的解是 $n < 8099$。也就是说，如果一个列中存储的数据小于 8099 字节，那么该列就不会成为溢出列，否则就会成为溢出列。不过，这个 8099 字节的限制只是针对只有一个列的 off_page_demo 表来说的。如果表中有多个列，则上面的不等式和结论就需要改一改了，所以重点就是：我们不用关注这个临界点是什么，只要知道如果一条记录的某个列中存储的数据占用的字节数非常多时，该列就可能成为溢出列。

4.3.5 DYNAMIC 行格式和 COMPRESSED 行格式

下面来看另外两个行格式：DYNAMIC 行格式和 COMPRESSED 行格式。我现在使用的 MySQL 版本是 5.7，它的默认行格式就是 DYNAMIC。这两个行格式与 COMPACT 行格式挺像，只不过在处理溢出列的数据时有点儿分歧：它们不会在记录的真实数据处存储该溢出列真实数据的前 768 字节，而是把该列的所有真实数据都存储到溢出页中，只在记录的真实数据处存储 20 字节大小的指向溢出页的地址（当然，这 20 字节还包括真实数据占用的字节数），如图 4-18 所示。

COMPRESSED 行格式不同于 DYNAMIC 行格式的一点是，COMPRESSED 行格式会采用

压缩算法对页面进行压缩，以节省空间。这里就不多唠叨了。

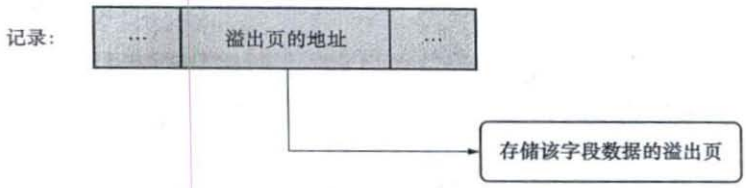

图 4-18 DYNAMIC 行格式和 COMPRESSED 行格式的溢出页

小贴士

REDUNDANT 是一种比较原始的行格式，它是非紧凑的。而 COMPACT、DYNAMIC 以及 COMPRESSED 行格式是较新的行格式，它们是紧凑的（占用的存储空间更少）。

4.4 总结

页是 InnoDB 中磁盘和内存交互的基本单位，也是 InnoDB 管理存储空间的基本单位，默认大小为 16KB。

指定和修改行格式的语法如下：

```
CREATE TABLE 表名 (列的信息) ROW_FORMAT=行格式名称;
ALTER TABLE 表名 ROW_FORMAT=行格式名称;
```

InnoDB 目前定义了 4 种行格式。

- COMPACT 行格式，如图 4-19 所示。

图 4-19 COMPACT 行格式示意图

- REDUNDANT 行格式，如图 4-20 所示。

图 4-20 REDUNDANT 行格式示意图

- DYNAMIC 和 COMPRESSED 行格式。

这两种行格式类似于 COMPACT 行格式，只不过在处理溢出列数据时有点儿分歧：它们不会在记录的真实数据处存储列真实数据的前 768 字节，而是把所有的数据都存储到所谓的溢出页中，只在记录的真实数据处存储指向这些溢出页的地址。另外，COMPRESSED 行格式会采用压缩算法对页面进行压缩。